Azamat Soliev

CS 499 Computer Science Capstone

Professional Self-Assessment

October 2026

I am completing the Bachelor of Science in Computer Science at Southern New Hampshire University, and I am preparing for a career as a full-stack software developer: someone who can take a problem from a conversation with the people who have it, through the database and the server, to the screen they will use. This ePortfolio is the evidence for that claim. It centers on one application, Azamat School Admin, a management platform for small language schools that I rebuilt across this capstone from Travlr Getaways, a travel booking site I created in CS 465. It was built for a real case: a teacher of mine who opened a small language school was managing students, schedules, attendance and payments by hand. The sections below describe the strengths the program developed in me, the professional values those strengths rest on, and how the pieces of this portfolio fit together.

Strengths Developed Across the Program

Collaborating in a Team Environment

My clearest team experience came in CS 250, Software Development Lifecycle, where I served as Scrum Master on the SNHU Travel project. I made sure the team's retrospectives were a safe space for honest feedback, and when the product direction shifted midway to wellness travel, my emails kept the Product Owner and the Tester aligned while I updated the affected user stories. That project taught me that a team moves at the speed of its shared understanding. I brought the same habit to this capstone, although it was individual work, by building it the way a team builds software: the code review was recorded for peers and a manager, each source file opens with a header that states its purpose, the repository keeps the complete history from the original submission to the final release, and every change is checked automatically by continuous integration before anyone else has to rely on it.

Communicating With Stakeholders

Several courses asked me to turn what a client said into something a developer could build. In CS 255, System Analysis and Design, I worked from the DriverPass interview transcript to write a business requirements document and a system design document with use case, activity, sequence and class diagrams, including role-based access for the owner, the IT administrator and the secretary. In CS 230 I evaluated operating platforms for The Gaming Room's multiplayer game and made recommendations based on the client's needs, and in CS 340 I built a dashboard for Grazioso Salvare that replaced manual spreadsheet review: staff can filter thousands of shelter records in seconds and see the results on a pie chart and a map. This capstone applies that lesson directly. Its reports present the same data three ways because the owner, the accountant and the teacher make different decisions: a class comparison for the owner, a reconcilable table with a spreadsheet export for the accountant, and an attendance checklist, with no money on it, for the teacher.

Data Structures and Algorithms

CS 300 taught me to choose a data structure for its performance rather than its familiarity: for a course planner, I compared a vector, a hash table and a binary search tree, and chose the tree because it keeps courses sorted while still searching efficiently. CS 370 taught me to debug a learning algorithm by separating code that is wrong from parameters that are wrong, when a reinforcement-learning agent that refused to move turned out to have been denied the chance to explore. The capstone's scheduling enhancement is where these came together. Instead of comparing a new lesson against a teacher's entire history, the conflict check seeks directly to the only window where a clash could occur, and I proved the window can never hold more than 32 lessons. In measurement, the naive approach examined all one million lessons in a teacher's history for every booking, while the bounded search examined about 22, and five thousand randomized cases confirmed that the two always reach the same verdict.

Software Engineering and Databases

Across the program I have built software in C++, Java, Python and JavaScript, against relational and document databases: JUnit-tested services with strict input validation in CS

320, a reusable Python data-access module for MongoDB in CS 340, an Android inventory application backed by SQLite in CS 360, and the original MEAN-stack Travlr application in CS 465. The capstone shows what that experience adds up to. In the original application, the trip, its only record type, needed twenty-one files of its own; now one configurable controller and one configurable screen serve all five of the school's record types, so adding a sixth means writing a schema and two short configuration entries. Five MongoDB aggregation pipelines produce the school's reports, with money summed in whole cents inside the database so that a settled balance is exactly zero. The work is supported by 214 backend tests, 69 interface tests and five end-to-end tests that drive a real browser through the running system. For the most important rules, I also broke the code on purpose to confirm that the tests would catch it, and that exercise exposed one test that could never have failed.

Security

CS 305 introduced me to security as an engineering discipline: for Artemis Financial I added a SHA-256 checksum service, configured HTTPS with a generated certificate, and used the OWASP Dependency-Check tool to confirm that the changes introduced no new vulnerabilities. This capstone made security a habit rather than a step. Every request is checked by role, by record ownership and by field, and teachers receive students' names but never their contact details. Passwords use PBKDF2 at 210,000 iterations, request bodies cannot smuggle in database operators, and simultaneous bookings cannot both slip past the conflict check. Reviewing my own earlier work also mattered. The final review found that a single sign-in attempt made while the database was unreachable could stop the entire server, a defect inherited from the original code. It also found that my own password upgrade from Enhancement One refused the very accounts it was written to protect, because its tests built those accounts differently from the way the database loads them. Both are fixed, and each is covered by a test that fails if the defect returns.

Professional Goals and Values

The program shaped three values I intend to keep. The first is to prove a claim rather than assert it: a performance claim gets a measurement, and a test is not finished until it has been shown to fail. The second is to design security in from the start, because the expensive

defects are the ones that fail silently. The third is honesty about limits. In CS 370 I wrote that I owe an organization honesty about the difference between a model that performed well in a demonstration and one that has actually been validated, and this portfolio follows that principle: its documentation lists what the system does not yet do, such as coordinating bookings across several servers, alongside what it does.

Building the portfolio has also made me more employable in a concrete way. Instead of listing courses, I can give an employer a public repository with its complete history, tests that run on every change, and written explanations of the main design decisions. My goal is a full-stack development role in which I can carry a feature from requirement to production, and I plan to strengthen that foundation with industry certifications after graduation.

How the Artifacts Fit Together

The artifacts in this portfolio are one application seen from four angles, in the order they were built. The code review, recorded before any enhancement, walks through Travlr Getaways as I submitted it for CS 465 and sets out the plan for each category. Enhancement One, software design and engineering, rebuilt the application as a role-based school platform around a generic controller and a configuration-driven interface. Enhancement Two, algorithms and data structures, added scheduling conflict detection with a proven bound on its cost. Enhancement Three, databases, added aggregation-pipeline reporting, exact money and database security. The final release retired the remaining travel features, closed public registration, rebuilt sign-in and connected every change to automated testing on a real MongoDB server.

Each enhancement depends on the one before it: the conflict check attaches to the controller built in Enhancement One, and the reports depend on both the role model and the scheduling data. Together they cover the full stack and all five course outcomes. The pages that follow present the code review, the original and final versions of the application side by side, and each enhancement with its narrative, its evidence and a link to the code exactly as it stood at that stage.

Course Outcomes

The five course outcomes of the program, and the specific skills in this portfolio that demonstrate each one:

Course outcome	Specific skills and where they are demonstrated
1. Collaborative environments. Employ strategies for building collaborative environments that enable diverse audiences to support organizational decision-making in the field of computer science.	Designing for several audiences: owner, accountant and teacher roles, each with access built for its own work, in Enhancement One, and reports organized around each role's decisions in Enhancement Three. Making work easy for others to review and build on: a code review recorded for peers and a manager, explanatory comments and a header on every source file, a tagged history, and automated checks on every change.
2. Professional communication. Design, develop, and deliver professional-quality oral, written, and visual communications that are coherent, technically sound, and appropriately adapted to specific audiences and contexts.	Adapting one message to different audiences and forms: the recorded code review (oral and visual), the narratives and this self-assessment (written), the diagrams and charts on each enhancement page (visual), a report format for each reader in Enhancement Three, and README, architecture, security and testing guides for developers.
3. Algorithms and trade-offs. Design and evaluate computing solutions that solve a given problem using algorithmic principles and computer science practices and standards appropriate to its solution, while managing the trade-offs involved in design choices.	Algorithm analysis and design: replacing an O(n) scan with an O(log n + k) bounded index search, proving that k never exceeds 32, measuring both with a benchmark, and checking them against each other on 5,000 random cases in Enhancement Two. Weighing index, money and data-model trade-offs in Enhancement Three.
4. Well-founded techniques. Demonstrate an ability to use well-founded and innovative techniques, skills, and tools in computing practices for the purpose of implementing computer solutions that deliver value and accomplish industry-specific goals.	Applying established patterns and tools: a generic controller factory and a configuration-driven interface in Enhancement One, compound indexes and a keyed lock in Enhancement Two, and MongoDB aggregation pipelines in Enhancement Three. Testing iteratively: 214 backend tests, 69 interface tests and five end-to-end tests, run by continuous integration on every change.
5. Security mindset. Develop a security mindset that anticipates adversarial exploits in software architecture and designs to expose potential vulnerabilities, mitigate design flaws, and ensure privacy and enhanced security of data and resources.	Finding and closing vulnerabilities: role, record and field checks and a stored cross-site scripting fix in Enhancement One; a booking race closed in Enhancement Two; defenses against query-operator and spreadsheet-formula injection, support for a least-privilege database account, and a names-only projection for teachers in Enhancement Three; and two defects found in review and fixed with tests in the final release.